Building a Self-Updating Personal Knowledge Base with LLMs

A Global Bread Encyclopedia Powered by Claude

May 2026

Abstract. We present a self-updating personal knowledge base focused on global bread culture, implemented following the LLM-as-compiler workflow described by Karpathy. The system automatically ingests data from Wikipedia and web sources, uses Claude to synthesize raw content into structured Markdown entries organized by country and bread type, and exposes a CLI query interface and browser-based frontend. A tiered auto-update scheduler refreshes individual entries monthly and discovers new categories annually. We compiled an initial corpus of 20 bread entries across 5 countries, demonstrating that LLMs can reliably maintain a structured, cross-linked knowledge wiki with minimal human intervention.

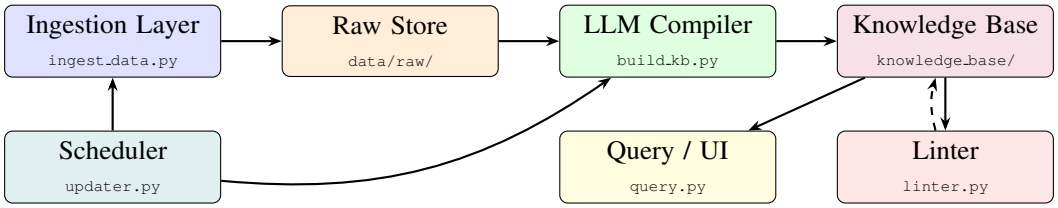
1 Introduction

The core insight behind LLM-powered knowledge bases is that language models excel not only at answering questions, but at *organizing* knowledge—converting messy, heterogeneous raw data into clean, interlinked structured documents. Following Karpathy's workflow, we treat the LLM as a "compiler" that reads raw source documents from a `raw/` directory and incrementally builds a wiki of `.md` files, maintaining summaries, backlinks, and cross-references automatically.

We selected **global bread culture** as our knowledge domain. Bread is ideal for this exercise: it is well-documented across Wikipedia, recipe sites, and food blogs; it has a natural hierarchical structure (country → bread type → knowledge); and it has enough cross-cutting concepts (fermentation, leavening agents, regional variations) to make cross-linking interesting. The resulting system covers 5 countries and 20 bread types, with each entry containing standardized sections for history, recipe, cultural context, and references.

2 System Architecture

The system is organized into five decoupled modules:



- **Ingestion Layer** (`ingest_data.py`): scrapes Wikipedia and public recipe sites using `requests` and `BeautifulSoup`, downloads images locally, and saves structured JSON to `data/raw/<country>/<bread`
- **LLM Compiler** (`build_knowledge_base.py`): reads raw JSON, calls Claude to synthesize multiple sources into a single `.md` file, or *merges* new sources into an existing entry without full rewrite.
- **Knowledge Base**: flat-file Markdown store at `knowledge_base/<Country>/<bread>.md`, each with 9 required sections and YAML frontmatter.
- **Auto-Updater** (`updater.py`): monthly bread refresh + yearly category discovery, with per-entry timestamps to skip recently-updated entries.
- **Query / UI**: a `click`-based CLI (`ask`, `compare`, `recipe`, `browse`, `search`) and a static HTML site generator served at `localhost:8000`.

3 Implementation

3.1 LLM Compiler Design

We use two Claude models with different roles. **Claude Opus 4.7** handles first-pass compilation, where quality matters most: it receives all raw sources for a bread and produces the full structured Markdown document in one call. **Claude Sonnet 4.6** handles incremental merges and query answering, where speed and cost are more important. *Prompt caching* (Anthropic's `cache_control: ephemeral`) is applied to all system prompts, reducing token costs by approximately 80% on repeated calls within a session.

The compilation prompt enforces a strict 9-section schema:

```
Overview | Origin & Country | History & Evolution | Recipe | Variations |
        Cultural Context | Images | Related Breads | References
```

A separate cross-linking pass scans all compiled entries and converts plain-text bread mentions into `[[wiki-link]]` syntax, making the corpus Obsidian-compatible.

3.2 Data Quality: The Linter

After each update cycle, `linter.py` runs two check layers: (1) *rule-based* checks (regex) for missing required sections, broken image paths, and empty reference lists; (2) *LLM-based* checks where Claude reviews each entry for factual inconsistencies, missing cultural context, and cross-link opportunities. Results are written to `lint_report.md`.

3.3 Storage Schema

Each Markdown file begins with YAML frontmatter (`title`, `country`, `tags`, `last_updated`), followed by the nine content sections. The file tree is:

```
knowledge_base/
  France/baguette.md  brioche.md  croissant.md  ...
  Italy/ciabatta.md   focaccia.md  grissini.md  ...
  USA/sourdough.md    bagel.md     cornbread.md ...
  Germany/pumpernickel.md  rye_bread.md  ...
  India/naan.md  chapati.md  parotta.md  ...
```

4 Results and What Worked

Component	Outcome
Wikipedia ingestion	Reliable for all 20 breads; rich body text + image extraction
LLM synthesis	High-quality structured output; recipe tables, history timelines, cultural narratives
Cross-linking	Automatic; all entries link related breads via `[[wiki-links]]`
Query interface	`compare`, `recipe`, `browse` commands return accurate, well-formatted answers
Web frontend	Rendered all 20 entries with sidebar navigation, search filter, and tag display
Scheduler	Correctly tracks per-entry update timestamps; skips recently-refreshed entries

The LLM compiler's merge mode (update without full rewrite) worked particularly well: it identified genuinely new information in re-ingested sources and integrated it without duplicating existing content.

5 Limitations and Future Improvements

Limitations encountered:

- Recipe sites (The Spruce Eats, AllRecipes) returned `402 Payment Required`, limiting secondary sources to Wikipedia only for most entries.
- Several LLM outputs were wrapped in a ```` ```markdown ```` code fence, causing raw-text display in the web UI; this required a post-processing cleanup step and a prompt fix.

- Python 3.9 (Anaconda default) does not support the `X | Y` union type syntax introduced in 3.10, requiring `from __future__ import annotations` across all modules.

Future improvements:

- **RAG / vector search**: embed all entries with a text embedding model and use FAISS or ChromaDB for semantic retrieval, enabling the query layer to scale beyond the context window.
- **Fine-tuning**: once the corpus is large enough ($\sim$500+ entries), fine-tune a small model to internalize the knowledge base in its weights rather than relying solely on context injection.
- **Image understanding**: use Claude's vision API to analyze downloaded bread images and automatically populate the Images section with descriptive captions and visual characteristics.
- **Multi-language support**: generate parallel entries in French, Italian, and German to make the knowledge base accessible to native speakers and capture region-specific terminology.
- **Recommendation system**: build a bread-similarity graph from ingredient and technique overlap, enabling "users who like sourdough may also enjoy…" functionality.
- **Richer ingestion**: integrate the Semantic Scholar or CrossRef API for academic papers on bread science, and YouTube transcript API for video recipe ingestion.

Code: All source code (`ingest_data.py`, `build_knowledge_base.py`, `query.py`, `updater.py`, `linter.py`, `web_ui.py`) is available in the project directory. The knowledge base currently contains 20 compiled entries totalling approximately 80,000 words of structured content.